

Employee Computer Monitoring Application

Developer requirements & scope document for an in-office productivity monitoring system deployed across 30 company-owned workstations.

Prepared for	Assigned Developer / Development Team
Deployment scale	30 employee workstations (single office)
Device ownership	Company-owned devices only
Document type	Requirements specification (v1.0 — baseline scope)
Status	Draft for developer review & estimation

1. Project Overview

This document specifies the requirements for a basic **employee computer monitoring application** for internal office use. The goal is to give management accurate visibility into workstation activity, productivity trends, and attendance across a team of 30 employees, all working on company-provided computers in a single office environment.

The system is intended for **transparent, disclosed workplace monitoring** — not covert surveillance. It should be deployed openly, with employees informed in advance (see Section 3). The developer should build the product around that assumption.

1.1 Objectives

- Track application and website usage to understand how work time is spent.
- Measure active vs. idle time and daily attendance (first login to last logout).
- Categorize activity as productive / neutral / unproductive for reporting.
- Give managers a clean web dashboard with per-employee and team-level reports.
- Keep the agent lightweight so it does not slow down employee machines.

1.2 Out of Scope (v1.0)

- Mobile phone or personal device monitoring — company desktops/laptops only.
- Full keystroke content logging (keylogging of typed text) — excluded for privacy and legal reasons; see Section 6.
- Remote screen control or remote desktop takeover.
- Reading personal email, chat message content, or private account credentials.

2. System Architecture

A standard three-part architecture: a lightweight **agent** installed on each workstation, a central **server / API** that receives and stores data, and a web-based **admin dashboard** for management.

Component	Runs on	Responsibility
Monitoring Agent	Each employee PC	Collects activity data (active app, window title, URL, idle/active state, login/logout), buffers locally when offline, and syncs to the server on a schedule.
Central Server / API	Company server or cloud VM	Receives agent data over HTTPS, authenticates agents, stores records in the database, runs productivity categorization, and serves the dashboard API.
Database	Alongside server	Stores employees, devices, activity logs, screenshots (if enabled), and settings.
Admin Dashboard	Browser (manager/HR)	Web UI for reports, per-employee drill-down, team summaries, and configuration.

2.1 Data flow

Agent collects data locally → batches it every 1–5 minutes → sends over HTTPS to the API → server validates & stores → dashboard reads from the database for reporting. If the network is down, the agent must queue data on disk and resend once connectivity returns (no data loss).

3. Legal, Privacy & Compliance Requirements

These requirements are **mandatory** and must be built into the product, not treated as optional. Workplace monitoring is legal in most jurisdictions **only when employees are informed** and the monitoring is proportionate to a legitimate business purpose.

Transparency is a hard requirement

- Employees must be notified in writing that monitoring is active (onboarding policy + signed acknowledgement).
- The agent must **not** be built to hide itself, evade task manager, or disguise its process. Covert/stealth spyware behaviour is out of scope.
- A visible indicator or entry in installed programs is recommended so the tool is discoverable.

3.1 Build-in compliance controls

- **Purpose limitation:** only collect data needed for productivity/attendance — no personal-account content.
- **Configurable scope:** admin can disable specific data types (e.g. turn screenshots off entirely).
- **Data retention:** configurable auto-deletion of old records (e.g. purge after 30/60/90 days).
- **Access control:** only authorized managers/HR can view data; every access is logged (audit trail).
- **Data protection:** Singapore's PDPA (and, if any staff or data sit elsewhere, laws like GDPR) apply — store data securely, limit access, and honour deletion requests.
- **No monitoring outside work context:** agent should respect work-hours settings where feasible.

Note for the business owner: before go-live, publish a written monitoring policy, get employee acknowledgement, and confirm the setup with a local employment-law professional. This document is a technical spec, not legal advice.

4. Functional Requirements

Core monitoring features the agent and server must support. Priority: **M** = must-have (v1.0), **S** = should-have, **C** = could-have (later).

Feature	Description	Pri.
Active application tracking	Record which app is in focus and for how long (e.g. Chrome 2h 10m, Excel 1h 30m).	M
Website / URL tracking	Capture domains/URLs visited in browsers with time spent per site.	M
Active vs. idle time	Detect keyboard/mouse inactivity beyond a threshold (e.g. 5 min) and mark time as idle.	M
Attendance / work hours	Log first login and last logout per day; compute total working time.	M
Productivity categorization	Tag apps/sites as Productive / Neutral / Unproductive via an admin-editable list.	M
Periodic screenshots	Optional, configurable interval (e.g. every 10 min). Must be toggleable and off by default.	S
Activity level metric	Keystroke/mouse counts as an activity score only — not the actual keys typed.	S
Application usage summary	Daily/weekly rollups per employee and per team.	M
USB / removable device log	Log when a USB storage device is connected (security/DLP signal).	C
Alerts	Optional flags, e.g. excessive idle time or access to blacklisted sites.	C

5. Admin Dashboard & Reporting

Web-based, accessible to authorized managers/HR only. Should cover:

- **Overview screen:** team-wide active time, average productivity %, who's online now, attendance snapshot.
- **Employee list:** all 30 staff with today's active time, idle time, and productivity score at a glance.
- **Employee detail:** timeline of apps/sites, active vs idle, screenshots (if enabled), day/week/month views.
- **Reports:** filter by date range, department, or individual; export to CSV / PDF.
- **Settings:** manage app/site categories, screenshot interval, retention period, and work hours.
- **User management:** create manager accounts, assign roles, view the access audit log.

6. User Roles & Permissions

Role	Permissions
Super Admin	Full access: settings, retention, user management, all reports, category rules.
Manager / HR	View reports and employee detail for assigned team; no system-config or user-management rights.
Auditor (optional)	Read-only access to the audit log and aggregate reports; cannot see individual screenshots.

Note on keystroke logging: v1.0 explicitly excludes capturing typed content. Only aggregate activity counts are permitted, to avoid capturing passwords, private messages, and other sensitive text.

7. Suggested Data Model

A minimal relational schema. The developer may adjust field names and add indexes as needed.

Table	Key fields
employees	id, name, email, department, status, created_at
devices	id, employee_id, hostname, os, agent_version, last_seen
activity_logs	id, device_id, app_name, window_title, url, start_time, end_time, is_idle
attendance	id, employee_id, date, first_login, last_logout, total_active_seconds
screenshots	id, device_id, captured_at, file_path (nullable — feature optional)
categories	id, pattern (app/domain), category (productive/neutral/unproductive)
users	id, email, password_hash, role, is_active
audit_log	id, user_id, action, target, timestamp, ip_address

8. Suggested Technology Stack

Recommendations only — the developer may substitute equivalents they are faster in.

Layer	Options
Agent (Windows)	C# / .NET (native Windows APIs), or Python packaged with PyInstaller. Runs as a background Windows service.
Server / API	Node.js (Express/NestJS) or Python (FastAPI/Django). REST API over HTTPS.
Database	PostgreSQL (recommended) or MySQL. SQLite only for a small pilot.
Dashboard	React or Vue front-end; or server-rendered (Django/Laravel) for speed.
Hosting	On-premise company server, or a private cloud VM (AWS/GCP/Azure) with restricted access.
Auth	JWT or session-based; hashed passwords (bcrypt/argon2); optional 2FA for admins.

9. Security Requirements

- All agent↔server traffic over **HTTPS/TLS**; reject plain HTTP.
- Each agent authenticates with a unique token; server rejects unknown agents.
- Passwords hashed (bcrypt/argon2); never stored in plain text.
- Role-based access control on every dashboard route and API endpoint.
- Encrypt sensitive data at rest (especially screenshots) and restrict file permissions.
- Full audit log of who viewed what and when.
- Rate-limit and validate all API inputs to prevent abuse/injection.

10. Non-Functional Requirements

Requirement	Target
Agent footprint	< 3% average CPU, low RAM; no noticeable slowdown for the employee.
Offline handling	Queue locally and resync — zero data loss on network drops.
Scale	Comfortably handle 30 agents; design to scale to ~100 without rework.

Requirement	Target
Dashboard speed	Report pages load in under ~3 seconds for a 30-person dataset.
Reliability	Agent auto-starts on boot and auto-recovers if it crashes.
Updatability	Support pushing agent updates without manually visiting each PC.

11. Deployment (30 Workstations)

- Provide a single installer (e.g. MSI) that installs the agent as an auto-start service.
- Support silent/scripted install so all 30 machines can be provisioned quickly (e.g. via GPO or a deployment script).
- Agent pulls its server address and token from a config file set at install time.
- Server deployed once (on-prem or cloud VM) with the database and dashboard.
- Provide an uninstaller and a documented rollback procedure.

12. Development Phases

Phase	Deliverable
Phase 1 — Core	Agent (app/URL/idle/attendance) + server + basic dashboard with per-employee reports.
Phase 2 — Reporting	Productivity categorization, team summaries, CSV/PDF export, roles & audit log.
Phase 3 — Optional	Screenshots, activity-score metric, USB logging, alerts, agent auto-update.

13. Deliverables & Acceptance Criteria

The developer should deliver:

- Agent installer + source code, and server/dashboard source code.
- Deployment guide (install server, roll out agents to 30 PCs) and admin user guide.
- Database schema and a short API reference.

Accepted when:

- All 30 agents report reliably and survive reboots and network drops.
- Dashboard shows accurate active/idle time, app/site usage, and attendance.
- Role-based access, audit logging, HTTPS, and configurable retention all work.
- Screenshots and activity metrics can be fully toggled off by an admin.

This is a baseline v1.0 specification intended for developer estimation and iteration. Confirm final feature scope and legal/compliance setup with the business owner and a local employment-law professional before deployment.